

Security Audit Report

Project Title

SecureCode Audit: Security Review and Remediation of a Python Authentication System

Author

Nirupa P

1. Introduction

Authentication systems are a primary target for cyberattacks because they control access to sensitive resources. Weak authentication mechanisms can lead to unauthorized access, credential theft, and brute-force attacks.

This project focuses on auditing a vulnerable login application, identifying security weaknesses, and implementing remediation measures using secure coding practices.

2. Project Objective

The objectives of this project are:

- Identify vulnerabilities in a login application.
- Analyze the security risks associated with each vulnerability.
- Implement appropriate security controls.
- Compare the vulnerable and secure versions of the application.
- Demonstrate secure coding practices.

3. Vulnerability Assessment

Vulnerability 1: Hardcoded Credentials

Description:

The username and password were directly stored within the source code.

Risk Level: High

Impact:

If an attacker gains access to the source code, valid credentials become immediately visible.

Vulnerability 2: Plaintext Password Storage

Description:

Passwords were stored and compared as plain text.

Risk Level: Critical

Impact:

Compromised source code would expose user credentials directly.

Vulnerability 3: Unlimited Login Attempts**Description:**

The application initially allowed unlimited authentication attempts.

Risk Level: High

Impact:

Attackers could perform brute-force attacks to guess credentials.

Vulnerability 4: Weak Password Acceptance**Description:**

The application accepted weak passwords without enforcing complexity requirements.

Risk Level: Medium

Impact:

Weak passwords significantly increase the likelihood of unauthorized access.

4. Security Controls Implemented**Password Hashing**

SHA-256 hashing was implemented to secure password storage and verification.

Benefit:

Passwords are no longer stored in plain text.

Login Attempt Limitation

The application restricts users to three authentication attempts.

Benefit:

Reduces the effectiveness of brute-force attacks.

Password Strength Validation

Passwords are required to contain:

- Minimum 8 characters
- Uppercase letter
- Lowercase letter
- Numeric value
- Special character

Benefit:

Improves credential strength and resistance against password attacks.

5. Before and After Comparison

<i>Security Feature</i>	<i>Vulnerable Version</i>	<i>Secure Version</i>
<i>Plaintext Password Storage</i>	Yes	No
<i>Password Hashing</i>	No	Yes
<i>Login Attempt Limiting</i>	No	Yes
<i>Password Complexity Rules</i>	No	Yes
<i>Brute Force Protection</i>	No	Yes

6. Results

The secure version of the application successfully mitigated multiple authentication-related vulnerabilities.

Implemented controls significantly improved the security posture of the application by protecting credentials, enforcing password complexity, and reducing the risk of brute-force attacks.

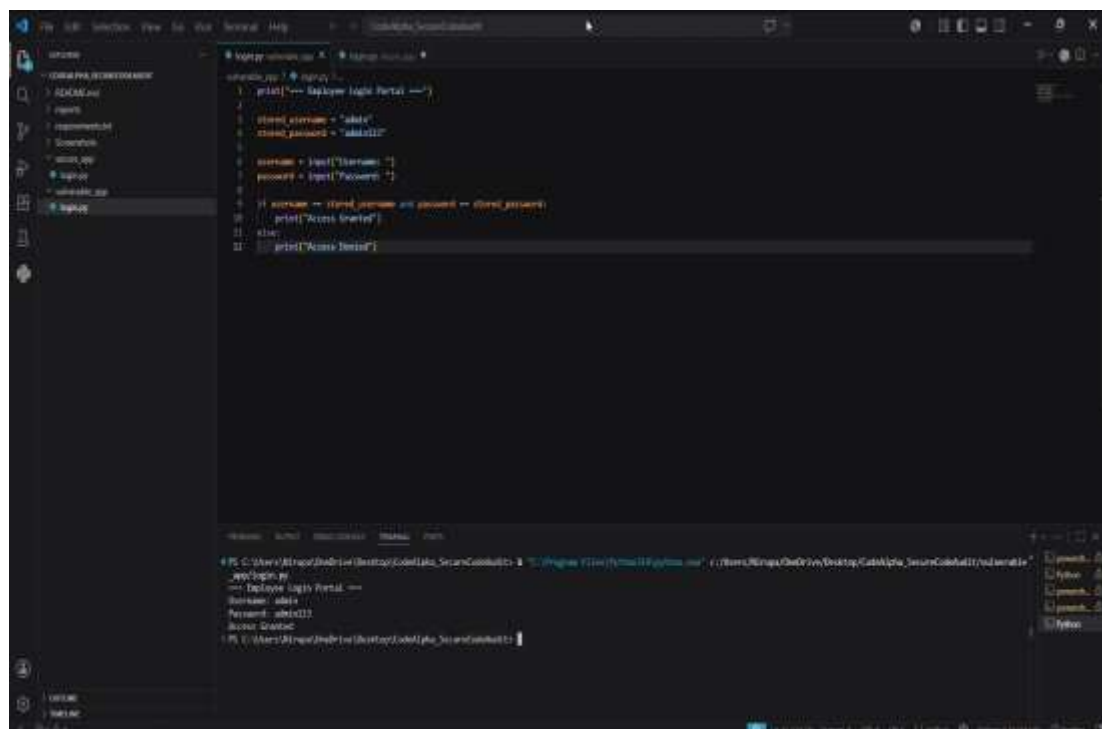
7. Screenshots

Include the following screenshots:

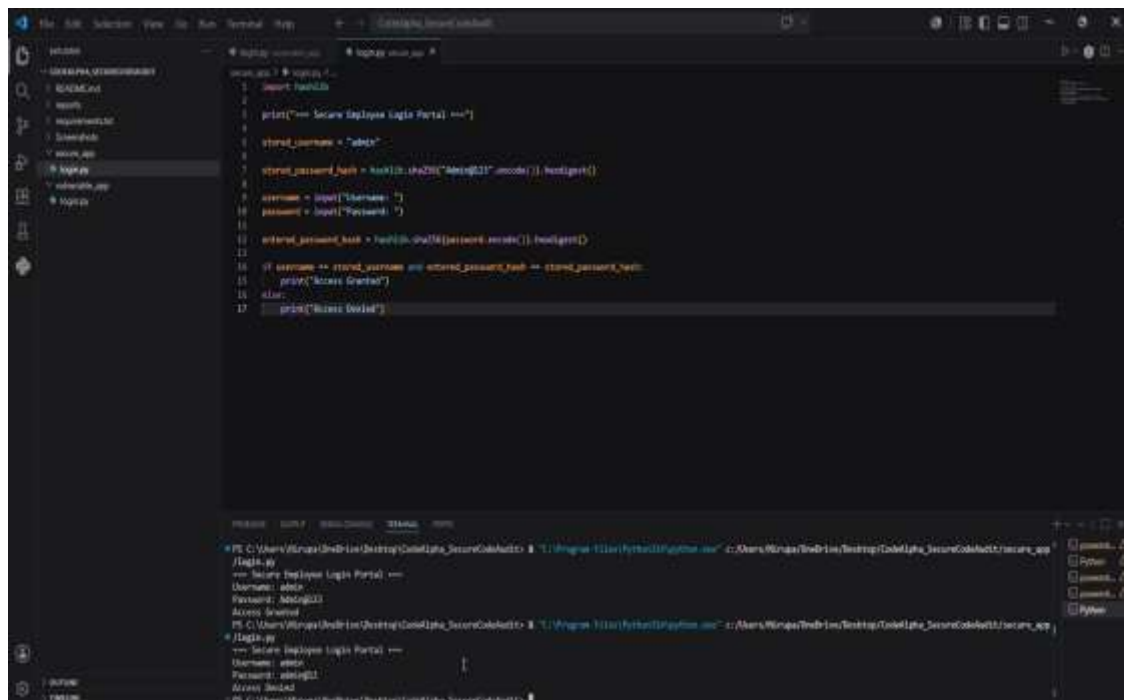
1. Source Code

```
login.py vulnerable_app login.py secure_app X
secure_app > login.py > ...
1 import hashlib
2
3 print("=== Secure Employee Login Portal ===")
4
5 stored_username = "admin"
6
7 stored_password_hash = hashlib.sha256("Admin@123".encode()).hexdigest()
8
9 username = input("Username: ")
10 password = input("Password: ")
11
12 entered_password_hash = hashlib.sha256(password.encode()).hexdigest()
13
14 if username == stored_username and entered_password_hash == stored_password_hash:
15     print("Access Granted")
16 else:
17     print("Access Denied")
```

2. Successful Login



3. Failed Login

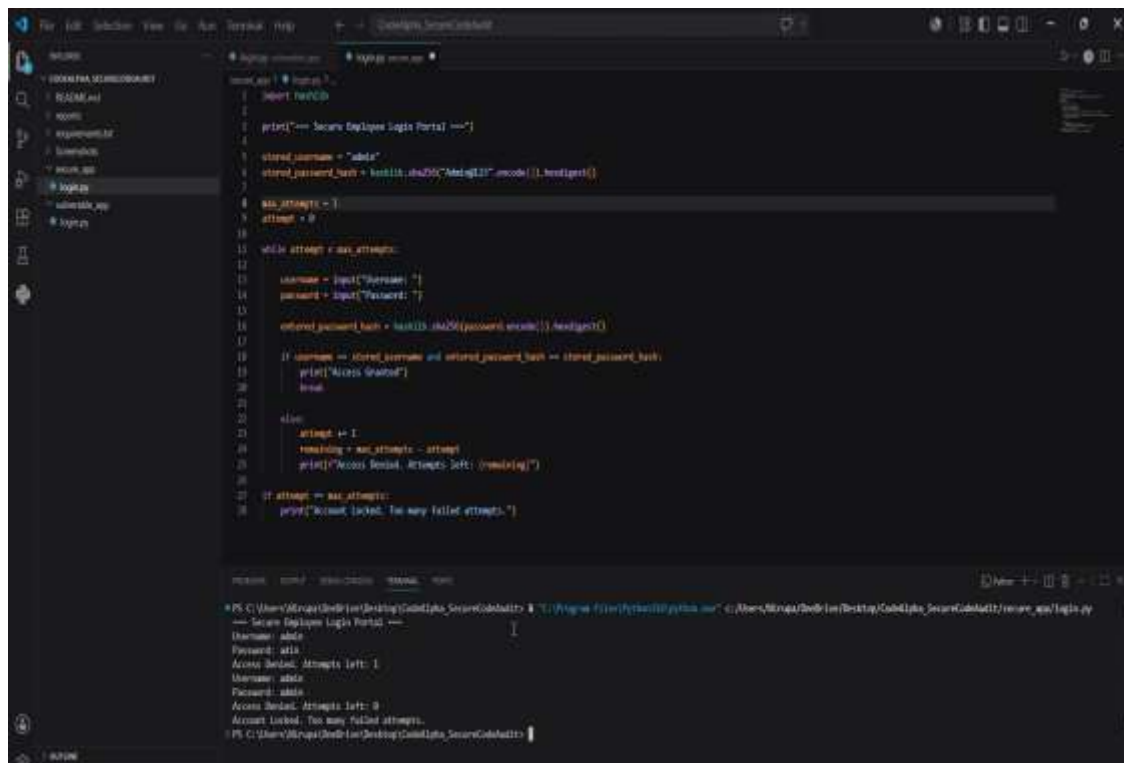


```
login.py
1 import hashlib
2
3 print("== Secure Employee Login Portal ==")
4
5 stored_username = "admin"
6
7 stored_password_hash = hashlib.sha256("Admin@123".encode()).hexdigest()
8
9 username = input("Username: ")
10 password = input("Password: ")
11
12 entered_password_hash = hashlib.sha256(password.encode()).hexdigest()
13
14 if username == stored_username and entered_password_hash == stored_password_hash:
15     print("Access Granted")
16 else:
17     print("Access Denied")
```

Terminal Output:

```
PS C:\Users\Nirupa\OneDrive\Desktop\Code\Python_SecurityCode\login> python .\login.py
== Secure Employee Login Portal ==
Username: admin
Password: Admin@123
Access Granted
PS C:\Users\Nirupa\OneDrive\Desktop\Code\Python_SecurityCode\login>
```

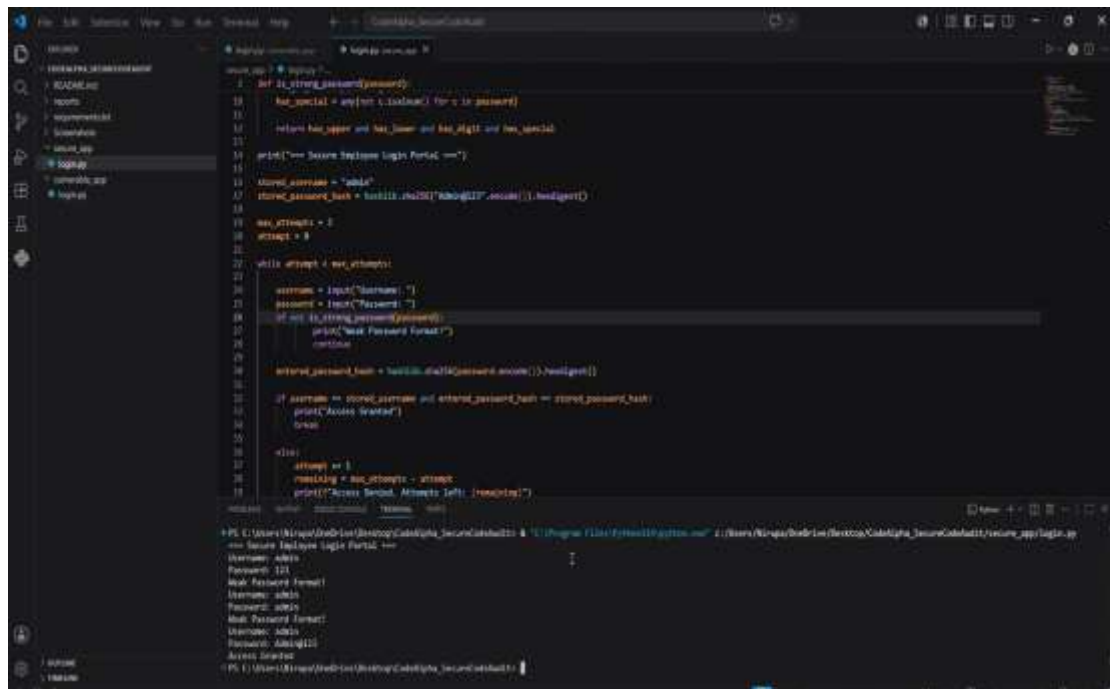
4. Account Locked After 2 Failed Attempts



```
login.py
1 import hashlib
2
3 print("== Secure Employee Login Portal ==")
4
5 stored_username = "admin"
6
7 stored_password_hash = hashlib.sha256("Admin@123".encode()).hexdigest()
8
9 max_attempts = 2
10 attempt = 0
11
12 while attempt < max_attempts:
13     username = input("Username: ")
14     password = input("Password: ")
15
16     entered_password_hash = hashlib.sha256(password.encode()).hexdigest()
17
18     if username == stored_username and entered_password_hash == stored_password_hash:
19         print("Access Granted")
20         break
21
22     else:
23         attempt += 1
24         remaining = max_attempts - attempt
25         print(f"Access Denied. Attempts left: {remaining}")
26
27 if attempt == max_attempts:
28     print("Account locked. Too many failed attempts.")
```

Terminal Output:

```
PS C:\Users\Nirupa\OneDrive\Desktop\Code\Python_SecurityCode\login> python .\login.py
== Secure Employee Login Portal ==
Username: admin
Password: admin
Access Denied. Attempts left: 1
Username: admin
Password: admin
Access Denied. Attempts left: 0
Account locked. Too many failed attempts.
PS C:\Users\Nirupa\OneDrive\Desktop\Code\Python_SecurityCode\login>
```



```
1 def is_string_password(password):
2     has_special = any(not c.isalnum() for c in password)
3
4     return has_upper and has_lower and has_digit and has_special
5
6 print("=== Secure Employee Login Portal ===")
7
8 stored_username = "admin"
9 stored_password_hash = hashlib.sha256(stored_password.encode()).hexdigest()
10
11 max_attempts = 3
12 attempts = 0
13
14 while attempts < max_attempts:
15     username = input("Username: ")
16     password = input("Password: ")
17     if not is_string_password(password):
18         print("Must Password Format!")
19         continue
20
21     entered_password_hash = hashlib.sha256(password.encode()).hexdigest()
22
23     if username == stored_username and entered_password_hash == stored_password_hash:
24         print("Access Granted")
25         break
26
27     attempts += 1
28     remaining = max_attempts - attempts
29     print("Access Denied. Attempts left: {remaining}")
30
31 print("==== End ====")
```

+ PS C:\Users\Niraj\Downloads (Desktop)\CodeAlpha_Secure> cd . & python .\login.py

=== Secure Employee Login Portal ===

Username: admin

Password: 123

Must Password Format!

Username: admin

Password: admin

Must Password Format!

Username: admin

Password: admin123

Access Granted

+ PS C:\Users\Niraj\Downloads (Desktop)\CodeAlpha_Secure> cd . & python .\login.py

8. Conclusion

This project demonstrated the process of identifying and remediating common authentication vulnerabilities in a Python-based login application.

The implementation of password hashing, brute-force protection, and password complexity validation improved the overall security of the system and provided practical exposure to secure coding principles and vulnerability assessment techniques.

The project highlights the importance of integrating security controls during software development to reduce risks and improve application resilience.